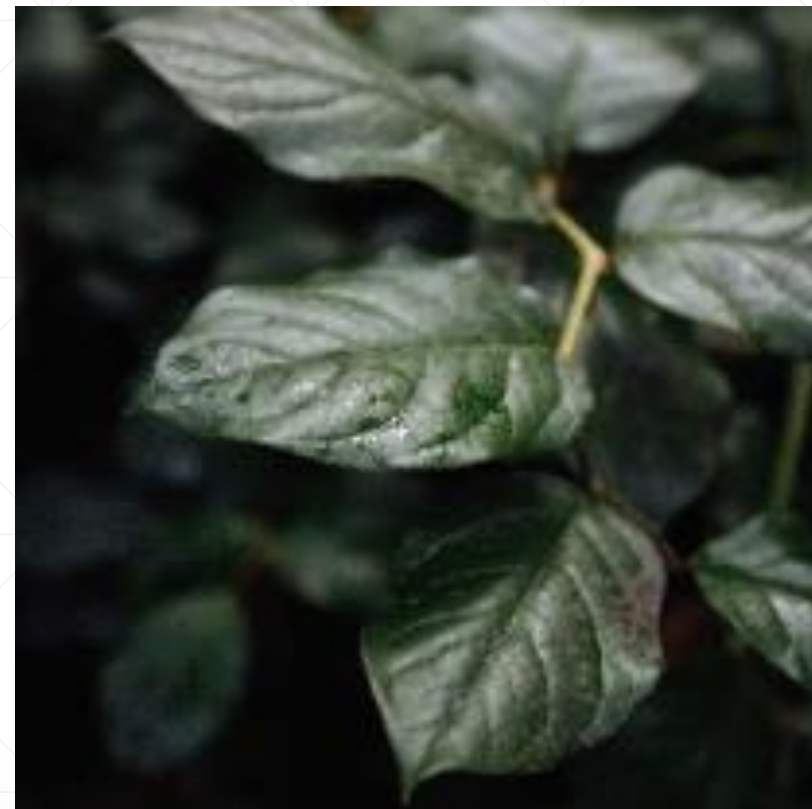




学会编写JavaFX Bean Property

北京理工大学计算机学院
金旭亮

概述



JavaFX Bean Property 类似于经典的 JavaBean Property 的编程方式，但它在内部使用了“JavaFX Properties and Binding Framework（JavaFX 属性与绑定框架）”来提供更强大的功能，以便应用于数据绑定场景。

JavaFX Bean Property的编程模式

JavaFX Bean property = getter + setter + property getter

示例：

```
private DoubleProperty height = new ...;  
  
public final double getHeight(){...}  
  
public final void setHeight(double h) {...}  
  
public DoubleProperty heightProperty() {...}
```



简单地说，定义一个JavaFX属性，需要定义四个成员.....

JavaFXBeanModel.java (1)

```
public class JavaFXBeanModel {  
  
    //整数型JavaFX Bean属性  
    7 usages  
    private IntegerProperty num =  
        new SimpleIntegerProperty(bean: this, name: "i", initialValue: 0);  
    1 usage  
    public final int getNum() { return num.get(); }  
    2 usages  
    public final void setNum(int num) { this.num.set(num); }  
    1 usage  
    public IntegerProperty numProperty() { return num; }  
}
```



自定义JavaFX Bean属性，内部其实封装的是SDK中所提供的SimpleIntegerProperty类。

JavaFXBeanModel.java (2)

//字符串型JavaFX Bean属性

7 usages

private StringProperty str =

new SimpleStringProperty(bean: this, name: "str", initialValue: "Hello");

1 usage

public final String getStr() { return str.get(); }

3 usages

public final void setStr(String str) { this.str.set(str); }

1 usage

public StringProperty strProperty() { return str; }

JavaFXBeanModel.java (3)

//对象型JavaFX Bean属性

7 usages

```
private ObjectProperty<Color> color =  
    new SimpleObjectProperty<>(  
        bean: this, name: "color", Color.BLACK);
```

no usages

```
public final Color getColor() { return color.get(); }
```

2 usages

```
public final void setColor(Color color) { this.color.set(color); }
```

1 usage

```
public ObjectProperty<Color> colorProperty() { return color; }
```



可以使用ObjectProperty<T>封装任何一个对象成为JavaFX Bean Property，本例中使用了Color对象。

测试自定义的JavaFX Bean属性-1

```
public static void main(String[] args) {  
    System.out.println("创建两个JavaFXBeanModel对象: obj1和obj2\n");  
    JavaFXBeanModel obj1 = new JavaFXBeanModel();  
    JavaFXBeanModel obj2 = new JavaFXBeanModel();  
  
    //构建一个监听器对象  
    ChangeListener listener = (prop, oldValue, newValue) -> {  
        System.out.println(prop.getClass().getSimpleName() + "属性值改变: 老值["  
            + oldValue + "],新值[" + newValue + "]);  
    };  
  
    System.out.println("为obj2的三个属性挂接ChangeListener");  
    //监听相应属性值的改变  
    obj2.num.addListener(listener);  
    obj2.str.addListener(listener);  
    obj2.color.addListener(listener);  
}
```

JavaFXBeanModel.java

测试自定义的JavaFX Bean属性-2

```
System.out.println("\n将obj1的三个属性绑定到obj2的相应属性上。 \n");
obj1.num.bind(obj2.num);
obj1.str.bind(obj2.str);
obj1.color.bind(obj2.color);

System.out.println("现在设置obj2的值: ");
obj2.setNum(100);
obj2.setStr("Modified");
obj2.setColor(Color.RED);
System.out.println("\n现在, obj2的值为: " + obj2);

System.out.println("\n因为obj1绑定到了obj2, 所以obj2的属性值一改, obj1的值也同步更新, 其值为: ");
System.out.println(obj1);
```

测试自定义的JavaFX Bean属性-3

测试结果表明，我们自定义的JavaFX Bean属性确实可以工作.....



```
Run
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" ...
创建两个JavaFXBeanModel对象: obj1和obj2

为obj2的三个属性挂接ChangeListener

将obj1的三个属性绑定到obj2的相应属性上。

现在设置obj2的值:
SimpleIntegerProperty属性值改变: 老值[0],新值[100]
SimpleStringProperty属性值改变: 老值[Hello],新值[Modified]
SimpleObjectProperty属性值改变: 老值[0x000000ff],新值[0xff0000ff]

现在, obj2的值为: JavaFXBeanModelExample [i=100, str=Modified, color=0xff0000ff]

因为obj1绑定到了obj2, 所以obj2的属性值一改, obj1的值也同步更新,其值为:
JavaFXBeanModelExample [i=100, str=Modified, color=0xff0000ff]

Process finished with exit code 0
```

扩充知识：JavaFX Bean属性对象的“延迟初始化”



JavaFX Bean 属性对象在内部需要实例化JavaFX数据绑定框架所提供的诸如IntegerProperty之类的对象，因此会占用一定的内存资源。



如果某JavaFX Bean有很多个JavaFX Bean属性对象，而在程序运行时，这些属性对象可能只有很少的几个被访问，这样一来，在JavaFX Bean创建时一次就全部创建其内部所有的属性对象，是不合适的——**用不着的你还浪费资源创建它干啥？！**



我们可以编写一些代码，仅在外界需要访问时才创建JavaFX Bean属性对象，这种编程方法，称为“**JavaFX Bean属性对象的延迟初始化**”。

延迟创建的JavaFX Bean Property

```
public class JavaFXBeanModelLazy {  
    //指定一个默认值  
    3 usages  
    private static final String DEFAULT_STR = "";  
    6 usages  
    private StringProperty str;  
  
    //尝试读取属性值，如果此属性已经创建，则返回其设置值  
    //否则，返回默认值  
    4 usages  
    public final String getStr() {  
        if (str != null) {  
            return str.get();  
        } else {  
            return DEFAULT_STR;  
        }  
    }  
}
```

图示为典型的“延迟创建”实现代码。

//仅当赋值时才创建JavaFX bean属性对象

3 usages

```
public final void setStr(String str) {  
    if ((this.str != null) || !(str.equals(DEFAULT_STR))) {  
        //调用strProperty()方法实例化JavaFX bean 属性对象  
        strProperty().set(str);  
    }  
}
```

//此方法负责创建JavaFX bean属性对象，并且保证只创建一次

1 usage

```
public StringProperty strProperty() {  
    if (str == null) {  
        System.out.println("JavaFX Bean属性对象真正地创建！");  
        str = new SimpleStringProperty(bean: this, name: "str", DEFAULT_STR);  
    }  
    return str;  
}
```

//测试延迟创建的JavaFX Bean属性

no usages

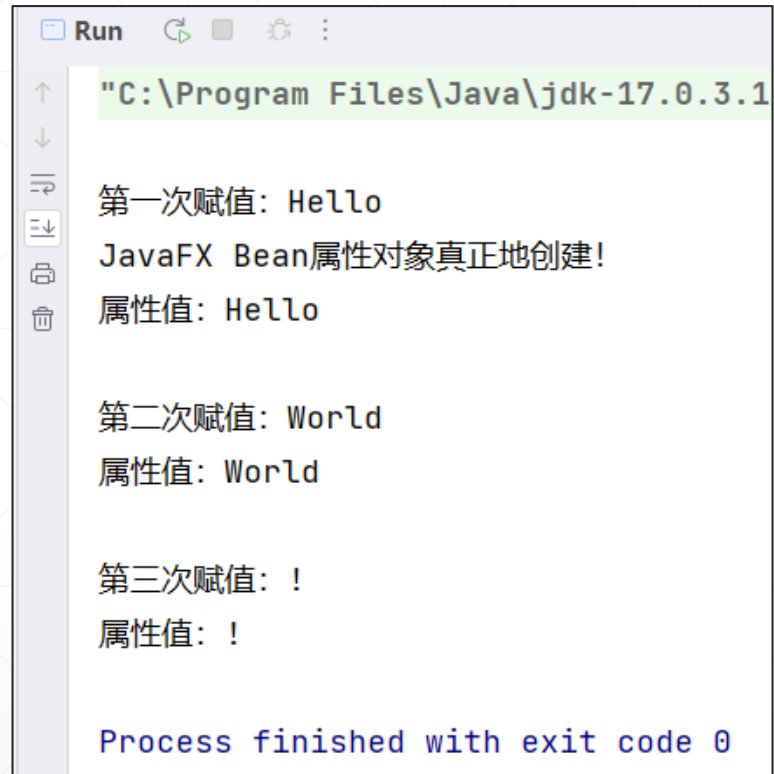
```
> public static void main(String[] args) {...}
```

```
}
```

测试延迟创建的JavaFX Bean属性

no usages

```
public static void main(String[] args) {  
  
    JavaFXBeanModelLazy lazy=new JavaFXBeanModelLazy();  
    System.out.println(lazy.getStr());  
    System.out.println("第一次赋值: Hello");  
    lazy.setStr("Hello");  
    System.out.println("属性值: "+lazy.getStr());  
  
    System.out.println("\n第二次赋值: World");  
    lazy.setStr("World");  
    System.out.println("属性值: "+lazy.getStr());  
  
    System.out.println("\n第三次赋值: !");  
    lazy.setStr("!");  
    System.out.println("属性值: "+lazy.getStr());  
}
```



```
Run  
"C:\Program Files\Java\jdk-17.0.3.1  
  
第一次赋值: Hello  
JavaFX Bean属性对象真正地创建!  
属性值: Hello  
  
第二次赋值: World  
属性值: World  
  
第三次赋值: !  
属性值: !  
  
Process finished with exit code 0
```

可以看到，仅仅在第一次赋值时才创建了属性对象，后面就重用它。如果不赋值，则永远不会创建属性对象。

典型编程模式—MVC

在实际开发中，经常使用MVC模式，其具体实现比较灵活，如果使用JavaFX Bean Property，则以下是一种典型的实现方案：

1

编写一个模型类JavaFXBeanModel，封装相应的数据，那些需要显示在UI界面上的数据，设计为JavaFX Bean Property。

2

编写一个视图类JavaFXBeanView，它代表用户界面，通常封装有各种UI控件。外界将JavaFXBeanModel注入进来，此类则将UI控件的相应属性绑定到JavaFXBeanModel中的JavaFX Bean Property。

3

编写一个控制器类 JavaFXBeanController，外界将JavaFXBeanModel和JavaFXBeanView注入进来，此类提供相应方法修改JavaFXBeanModel中的JavaFX Bean Property值，然后JavaFXBeanView中的控件将会自动刷新。

视图类示例

```
public class JavaFXBeanView {  
    //包容的模型对象  
    4 usages  
    private JavaFXBeanModel model;  
  
    1 usage  
    public JavaFXBeanView(JavaFXBeanModel model) {  
        this.model = model;  
        hookupChangeListeners();  
    }  
  
    //当Model的属性值更改时, 此方法可以实现UI界面的刷新  
    1 usage  
    private void hookupChangeListeners() {...}  
}
```

本例只是简单地附加了监听器，在真实项目中，这里可以在完成特定的数据处理任务后，绑定各种UI控件显示数据处理结果。

```
private void hookupChangeListeners() {  
    model.numProperty().addListener(  
        (observable, oldValue, newValue)  
        -> System.out.println("num属性值更改: 老值 = " + oldValue  
        + ", 新值 = " + newValue));  
  
    model.strProperty().addListener(  
        (observableValue, oldValue, newValue)  
        -> System.out.println("str属性值更改: 老值 = " + oldValue  
        + ", 新值 = " + newValue));  
  
    model.colorProperty().addListener(  
        (observableValue, oldValue, newValue)  
        -> System.out.println("color属性值更改: 老值 = " + oldValue  
        + ", 新值 = " + newValue));  
}
```

控制器示例

```
public class JavaFXBeanController {  
    //模型  
    7 usages  
    private JavaFXBeanModel model;  
    //视图  
    1 usage  
    private JavaFXBeanView view;  
    1 usage  
    public JavaFXBeanController(JavaFXBeanModel model,  
                                JavaFXBeanView view) {...}  
  
    //修改整数属性的值  
    1 usage  
    public void incrementIPropertyOnModel() {  
        model.setNum(model.getNum() + 1);  
    }  
    //修改字符串属性的值  
    1 usage  
    public void changeStrPropertyOnModel() {...}  
    // 修改颜色属性的值  
    1 usage  
    public void switchColorPropertyOnModel() {...}  
}
```

//修改字符串属性的值

1 usage

```
public void changeStrPropertyOnModel() {  
    final String str = model.getStr();  
    if (str.equals("Hello")) {  
        model.setStr("World");  
    } else {  
        model.setStr("Hello");  
    }  
}
```

// 修改颜色属性的值

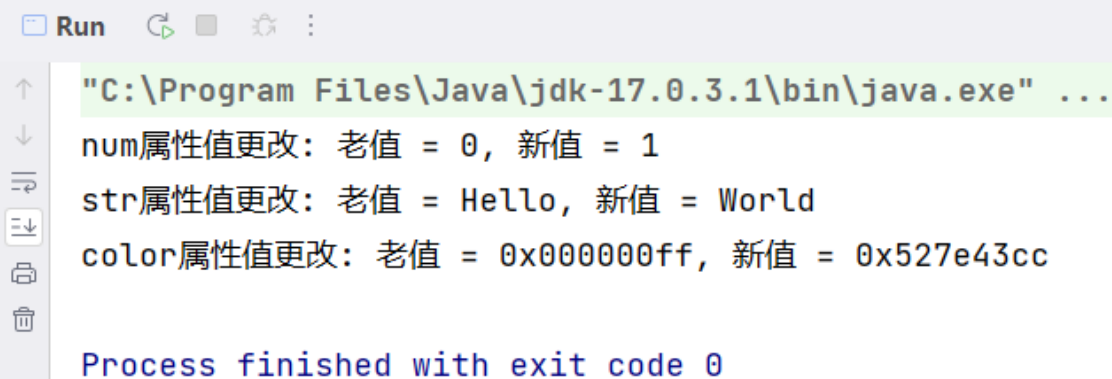
1 usage

```
public void switchColorPropertyOnModel() {  
    Random random = new Random();  
    Color color = new Color(  
        random.nextDouble(),  
        random.nextDouble(),  
        random.nextDouble(), opacity: 0.8);  
    model.setColor(color);  
}
```

测试代码

```
public class JavaFXBeanMVCTest {  
  
    no usages  
    public static void main(String[] args) {  
        //创建模型和视图对象，将其“注入到控制器中”  
        JavaFXBeanModel model = new JavaFXBeanModel();  
        JavaFXBeanView view = new JavaFXBeanView(model);  
        JavaFXBeanController controller =  
            new JavaFXBeanController(model, view);  
  
        //控制器修改模型对象的属性值，视图对象将得到通知  
        controller.incrementIPropertyOnModel();  
        controller.changeStrPropertyOnModel();  
        controller.switchColorPropertyOnModel();  
    }  
}
```

运行结果：



```
Run  
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" ...  
num属性值更改：老值 = 0, 新值 = 1  
str属性值更改：老值 = Hello, 新值 = World  
color属性值更改：老值 = 0x000000ff, 新值 = 0x527e43cc  
  
Process finished with exit code 0
```

小结



本部分展示了如何基于JavaFX SDK中所提供的基础设施之上，构建出自己的数据实体类。



在实际开发中，我们经常将要显示给用户看的数据封装为数据实体类，然后将其绑定到UI控件之上，请注意这个思路的一种具体实现方式——MVC。



下一讲，我们就来看看JavaFX应用中是怎样显示数据实体对象中所包容的数据的。